Indexes in MySQL

Indexes are one of the most important concepts in query optimization. They help MySQL fetch records faster without scanning the complete table. Most slow APIs in backend systems are caused by: • Missing indexes
• Poor query design
• Full Table Scans
• Unoptimized filtering queries

What is an Index?

Indexes are separate lookup structures created on one or more columns of a table.

Without indexes:
• MySQL scans rows one by one

With indexes:
• MySQL directly jumps to matching rows

Indexes work similar to a book index where you directly jump to the required topic instead of reading every page.

Real-Time Scenario — Instagram User Search

Instagram-like systems frequently search users by username.

Without indexes, searching millions of users becomes slow.

```
SELECT * FROM users
WHERE username='surya_mahesh';
```

Scenario:
When a user searches for an Instagram profile, the backend filters records using the username column.

Indexing username makes this search extremely fast even with crores of users.

Creating Index on Username

Create index on username column.

```
CREATE INDEX idx_username
ON users(username);
```

Now MySQL can directly locate matching usernames instead of scanning the entire table.

Real-Time Scenario — Library Management System

Libraries frequently search books using ISBN numbers.

```
SELECT * FROM books
WHERE isbn='9789332575109';
```

Scenario:
When a librarian scans or searches a book using ISBN, indexing helps retrieve the book instantly.

Creating ISBN Index

Indexing ISBN improves search speed significantly.

```
CREATE INDEX idx_isbn
ON books(isbn);
```

ISBN searches become much faster because MySQL avoids Full Table Scans.

Real-Time Scenario — Swiggy Restaurant Search

Food delivery platforms frequently filter restaurants using city and rating.

```
SELECT * FROM restaurants
WHERE city='Bangalore'
AND rating >= 4.5;
```

Scenario:
Users searching restaurants in Bangalore with high ratings should receive fast responses.

Composite indexes optimize such multi-column filtering.

Composite Index

Composite indexes are created on multiple columns.

```
CREATE INDEX idx_city_rating
ON restaurants(city, rating);
```

Now MySQL can optimize filtering using both city and rating together.

Real-Time Scenario — Instagram Feed

Instagram-like systems fetch latest posts for users.

```
SELECT * FROM posts
WHERE user_id=101
ORDER BY created_at DESC;
```

Scenario:
Fetching latest posts quickly is critical for feed loading performance.

Optimizing Feed Query

Composite indexes improve filtering + sorting together.

```
CREATE INDEX idx_user_created
ON posts(user_id, created_at);
```

This helps MySQL optimize:
• WHERE filtering
• ORDER BY sorting
together efficiently.

How MySQL Uses Indexes Internally

One of the biggest beginner confusions is:
'We created indexes, but where are we actually using them?'

Answer:
MySQL Query Optimizer automatically decides when to use indexes.

```
EXPLAIN
SELECT * FROM users
WHERE email='surya@gmail.com';
```

EXPLAIN helps identify:
• Whether index is used
• Which index is used
• Number of rows scanned
• Query execution strategy

When Indexes May NOT Be Used

Functions on indexed columns can break optimization.

```
SELECT * FROM users
WHERE YEAR(created_at)=2025;
```

Applying functions on indexed columns often prevents MySQL from efficiently using indexes.

Better Optimized Query

Range filtering allows MySQL to use indexes properly.

```
SELECT * FROM users
WHERE created_at >= '2025-01-01'
AND created_at < '2026-01-01';
```

This approach is much more optimized for large-scale systems.

When NOT to Overuse Indexes

Too many indexes can slow:
• INSERT operations

• UPDATE operations

• DELETE operations

Indexes improve READ performance but affect WRITE performance.

Backend Developer Perspective

Indexes are heavily used in:

- Spring Boot APIs
- Authentication systems
- Analytics dashboards
- Food delivery apps
- Social media platforms
- E-commerce systems

Most slow APIs are actually caused by poor database optimization.

Key Takeaways

- Indexes reduce Full Table Scans
- MySQL automatically decides when to use indexes
- Composite indexes optimize multi-column filtering
- EXPLAIN helps analyze query execution
- Proper indexing is critical for scalable backend systems

Indexes Performance Comparison

Feature	Without Index	With Index
Rows Scanned	All Rows	Matching Rows Only
Execution Time	Higher	Lower
Performance	Slow	Fast
Scalability	Poor	Better